Your 3-Day Mentor Guide: Materials, Links & Checkpoints

This is the resourced, "sit next to you" version of your bootcamp plan. Each block has: what to learn, exactly where to learn it, what to build, and a checkpoint so you know you're ready to move on. Work through it top to bottom — don't skip the checkpoints, they're what keep this from becoming "watched a video, learned nothing."

How to use this doc: open each link in a new tab, watch/read at 1.25x–1.5x speed if it's a video, then close the tab and *type the code yourself* — don't copy-paste. Typing it is what builds the muscle memory.

DAY 1 — Foundations + CLI Automation Tool

Module 1.1 — Syntax refresher (skip sections you're already solid on)

- Official Python tutorial (best written reference): https://docs.python.org/3/tutorial/
- If you want a fast video refresher instead: freeCodeCamp's Python course page (links to the full YouTube video): https://www.freecodecamp.org/news/python-programming-course/

Checkpoint: you can write a function with default arguments, a list comprehension, and a `for`/`else` loop without looking it up.

Module 1.2 — Error handling

- Official docs on exceptions: https://docs.python.org/3/tutorial/errors.html

Checkpoint: you can explain the difference between `except Exception` and `except FileNotFoundError`, and you've written one custom exception class (`class InvalidExpenseError(Exception): pass`).

Module 1.3 — File I/O with CSV

- Official `csv` module docs: https://docs.python.org/3/library/csv.html

Checkpoint: you can read a CSV into a list of dicts using `csv.DictReader` and write one back with `csv.DictWriter`.

Module 1.4 — argparse (build your CLI interface)

- Official argparse tutorial (read this first, it's short and excellent):

https://docs.python.org/3/howto/argparse.html

- Video walkthrough: https://www.youtube.com/watch?v=0twL6MXCLdQ

Checkpoint: `python your_script.py --help` prints a clean usage message, and `python your_script.py add "coffee" 4.50` correctly parses a subcommand and two arguments.

Module 1.5 — Logging (swap your prints for this)

- Official logging how-to: https://docs.python.org/3/howto/logging.html

Checkpoint: your script writes an `app.log` file and logs at least one `INFO` and one `ERROR` message.

BUILD: Project 1 (2-4 hours)

Pick the Expense Tracker CLI or File Organizer from your original plan. Requirements recap:

- `argparse` subcommands (`add`, `list`, `summary` or similar)
- Data persisted in CSV
- At least one custom exception, handled gracefully
- Logging instead of print for status/errors

Module 1.6 — Git & GitHub (do this alongside building, commit as you go)

- Video: Git Tutorial for Beginners — Command-Line Fundamentals (Corey Schafer, 31 min): https://www.youtube.com/watch?v=HVsySz-h9r4
- Writing a good README: https://www.makeareadme.com/

End-of-day checkpoint: repo is on GitHub, has 5+ commits (not one dump), and a README with install/run instructions and a screenshot.

DAY 2 — OOP + APIs + Data Project

Module 2.1 — Object-Oriented Programming

Watch in order (Corey Schafer's OOP series — the standard, still excellent):

1. Classes and Instances: https://youtu.be/ZDa-Z5JzLYM
2. Class Variables: https://youtu.be/BJ-VvGyQxho
3. Classmethods and Staticmethods: https://youtu.be/rq8cL2XMM5M
4. Inheritance: https://youtu.be/RSl87lqOXDE

5. Special/Dunder Methods (`__str__`, `__repr__`): https://youtu.be/3ohzBxoFHAY Full playlist: https://www.youtube.com/playlist?list=PL-osiE80TeTsqhIuOqKhwlXsIBIdSeYtc

Checkpoint: you can write a class with `__init__`, one instance method, one classmethod, a subclass that overrides a method, and a working `__repr__`.

Module 2.2 — Working with APIs (`requests`)

- Real Python's Requests guide (the best single reference): https://realpython.com/python-requests/
- Real Python on consuming REST APIs specifically: https://realpython.com/api-integration-in-python/

Checkpoint: you can hit a public API, check `response.status_code`, call `.raise_for_status()`, and parse `.json()` into a Python dict — with a `try/except` around network errors.

Module 2.3 — Local persistence with SQLite

- Official `sqlite3` docs: https://docs.python.org/3/library/sqlite3.html

Checkpoint: you can create a table, insert a row, and query it back using `sqlite3` from Python.

BUILD: Project 2, part 1 (3-4 hours)

Build your API-driven OOP tool (weather dashboard, repo inspector, etc.) per your original spec: 2+ classes, error-handled API calls, data saved to SQLite or JSON.

Module 2.4 — Testing with pytest + mocking

- pytest full course (freeCodeCamp): https://www.youtube.com/watch?v=cHYq1MRoyI0
- Mocking explained (Real Python — read this one, it's the clearest): https://realpython.com/python-mock-library/
- Mocking video if you prefer watching: https://www.youtube.com/watch?v=wZ_WXhwVRG0

Checkpoint: you have a `tests/` folder with at least 3 passing tests, and at least one test mocks the API call (`unittest.mock.patch`) so it runs without internet access.

End-of-day checkpoint: Project 2 pushed to GitHub with a README explaining your class design, plus a passing test suite.

DAY 3 — Web Backend + Capstone

Module 3.1 — REST fundamentals + FastAPI

- Official FastAPI tutorial (the canonical, extremely well-written docs — bookmark this): https://fastapi.tiangolo.com/learn/
- Video crash course (freeCodeCamp, ~1 hr): https://www.youtube.com/watch?v=tLKKmouUams

Checkpoint: a minimal `GET /` endpoint runs locally with `uvicorn main:app --reload`, and you can see the auto-generated docs at `/docs`.

Module 3.2 — Pydantic models + CRUD + SQLite

Covered inside the FastAPI docs tutorial above (sections on request bodies and path/query params). For the database side, reuse what you learned with `sqlite3` on Day 2, or step up to SQLAlchemy if you have time — the FastAPI docs have a section on SQL databases linked from the same learn page.

Checkpoint: you have working `POST`, `GET` (list + by id), `PUT`, and `DELETE` endpoints, all validated with Pydantic models, all visible and testable from `/docs`.

BUILD: Project 3 — Capstone (4-5 hours)

Task Manager / Job Application Tracker API per your original spec. Must-haves: clear folder structure, `requirements.txt`, a few endpoint tests using FastAPI's `TestClient` (same pytest skills from Day 2).

Module 3.3 — Deploy it live

- Official Render guide for FastAPI: https://render.com/docs/deploy-fastapi
- Step-by-step video walkthrough: https://www.youtube.com/watch?v=plcaPeJXeM8

Checkpoint: your API has a live `https://yourproject.onrender.com/docs` URL you can click from your phone.

Module 3.4 — Polish: profile README + all 3 repo READMEs

- GitHub's own guide to profile READMEs: https://docs.github.com/en/account-and-

profile/how-tos/profile-customization/managing-your-profile-readme

- General README best practices (applies to your 3 project repos too): https://www.makeareadme.com/

Final checkpoint (this is the one that matters most):

- [ ] 3 repos, pinned on your profile
- [ ] Each has a README with what/why/how + screenshot or GIF
- [ ] Profile README (repo named exactly your GitHub username) with a short bio + links to your top 3 projects
- [ ] Capstone has a live deployed link
- [ ] Resume has 3 impact-style bullets, one per project

If you get stuck (mentor notes)

- **Google the exact error message first**, in quotes. 90% of Python errors have a Stack Overflow answer.
- If a video moves too fast, drop to 1x speed and pause to actually type the code — don't just watch.
- If a whole day feels too packed: it's fine to spill Day 3 into a Day 4. A finished, well-documented capstone a day late beats a rushed, broken one on time.
- Stuck for more than 20-30 minutes on one bug? That's the moment to ask me — paste the exact error and the relevant code, and I'll help you debug it live.